

Technical Architecture Blueprint: Modular E2EE Messaging Platform

1. Project Philosophy & Core Architecture

The platform architecture follows a strict separation between mechanism and policy, mimicking the foundational design of modern operating system kernels:

- **Core C++ Engine ("The Kernel"):** A headless, high-performance C++20 engine that handles raw network sockets, protocol parsing, cryptographic session ratchets, local state caching, and persistent encrypted storage. It maintains no UI code and exposes a deterministic C-ABI (`extern "C"`) interface for binding to higher-level environments.
- **Frontends ("Distros"):** Decoupled applications spanning Graphical interfaces (GTK, Qt, Flutter, Godot, Electron) and Terminal interfaces (FTXUI, ncurses). Frontends dynamically bind to the C++ shared library (`.so`, `.dll`, `.dylib`), consuming state streams via lock-free event queues and emitting user actions back to the core engine. Frontends never access raw cryptographic key material or manage wire connections directly.

2. C++ Build System & Dependency Stack

Standard & Build Infrastructure

- **C++ Standard:** C++20, leaning heavily on `<format>`, concepts for explicit C-ABI boundary mapping, coroutines for non-blocking asynchronous flow, and `std::jthread` for RAI thread management.
- **Build System:** CMake 3.25+ utilizing modern target-based semantics and multi-config generators (Ninja / MSVC).
- **Package Manager:** Conan 2.0 or vcpkg in manifest mode to enforce hermetic, reproducible builds across Linux (x86_64, ARM64), macOS (universal binaries), and Windows.

Recommended C++ Dependencies

Component	Library	Rationale
Networking	<code>asio</code> (standalone) or <code>uWebSockets</code>	Header-only asynchronous I/O; sub-millisecond network event resolution with footprint < 5MB
Cryptography	<code>libsodium</code> / <code>OpenSSL 3.x</code>	Constant-time, battle-tested cryptographic primitives (X25519, Ed25519, AES-256-GCM, ChaCha20-Poly1305)
Serialization	<code>nlohmann_json</code> or <code>FlatBuffers</code>	JSON used for human-readable diagnostic logging; FlatBuffers for zero-copy binary IPC overhead
Database	<code>sqlite3</code> + <code>sqlite_orm</code>	Zero-configuration embedded state engine utilizing Write-Ahead Logging (WAL) for multi-threaded read performance
Logging	<code>spdlog</code>	High-throughput, thread-safe asynchronous logging overhead wrapped in zero-cost macros
Concurrency	<code>concurrentqueue</code> (moodycamel)	Lock-free lockless single-producer/single-consumer and multi-producer queues for inter-thread messaging

3. Server Daemon Architecture ("Deploy Anywhere")

The server component acts as an un-trusted, minimalist router designed to run reliably on resource-constrained host environments.

Resource Footprint & Compatibility

- **Memory Overhead:** < 30 MB idle memory footprint, low CPU footprint under heavy I/O. Compiled as a single, static binary.
- **Deployment Targets:** Raspberry Pi 4/5 (ARM64), \$3/month cloud VPS instances, low-power NAS containers, and repurposed legacy hardware.

Role as Zero-Trust Relay

- **Blind Routing:** The server handles opaque encrypted payload envelopes. It lacks both the keys and logic required to inspect message contents or metadata beyond routing tokens.
- **Store-and-Forward:** Implements a mailbox model for offline clients, queuing encrypted payloads until recipient delivery is acknowledged.
- **Access Enforcement:** Validates network-level Access Control Lists (ACLs), tracks active channel memberships, and enforces anti-abuse rate limits.

Storage Layer

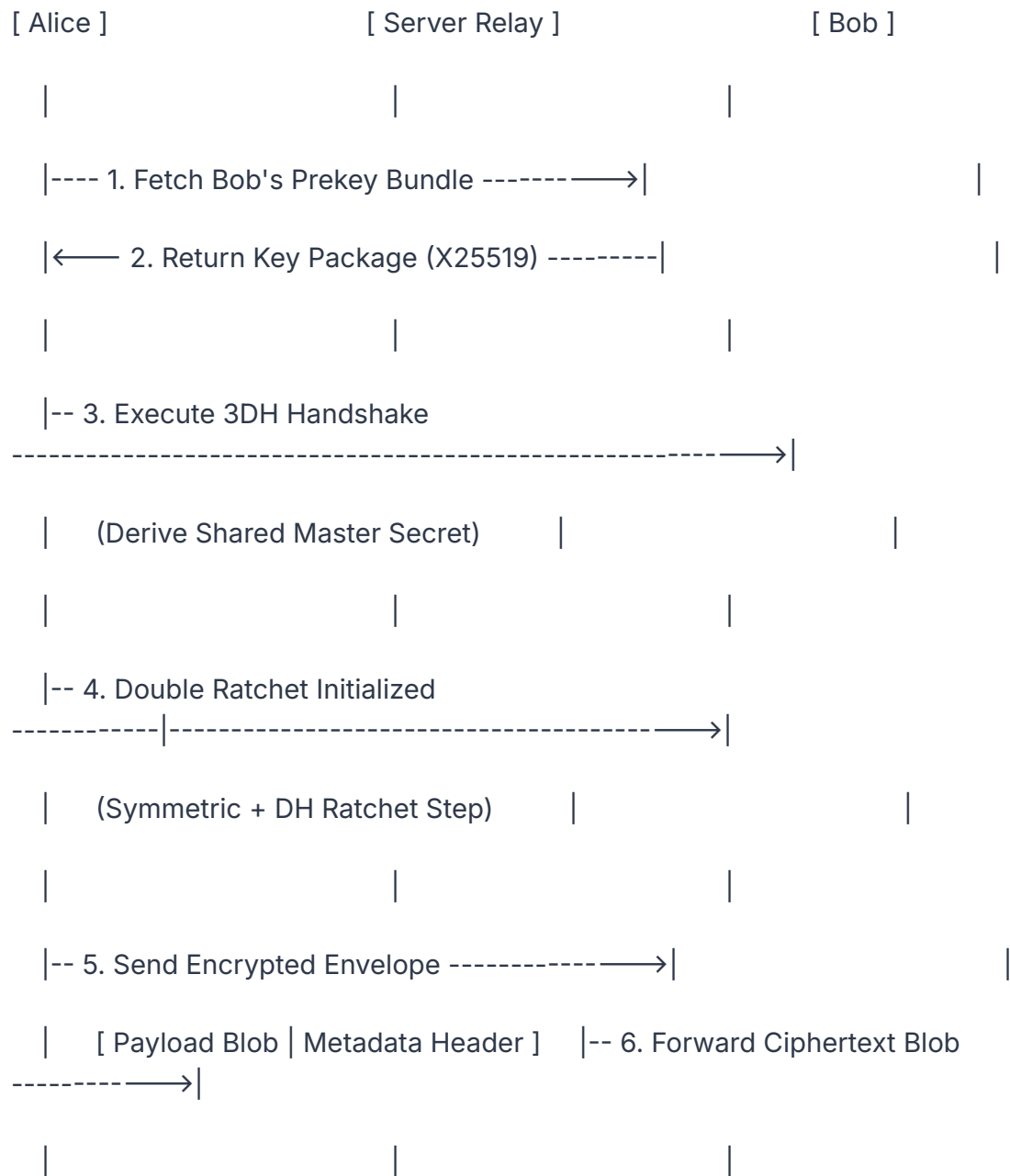
Driven by an embedded SQLite3 instance running in Write-Ahead Logging (WAL) mode. The schema manages user public key registries, room permission mapping, invite token lifetimes, and persistent ciphertext queues.

4. End-to-End Cryptography Engine

Cryptographic Primitives

- **Identity Keys:** Ed25519 for digital signatures and peer identity authentication; X25519 for ephemeral key exchange.
- **Symmetric Cipher:** AES-256-GCM or ChaCha20-Poly1305 (with hardware accelerated AES-NI fallback) providing Authenticated Encryption with Associated Data (AEAD).
- **Key Derivation:** HKDF (HMAC-SHA256 Extract-and-Expand) for inner ratchet steps; Argon2id for local passphrase derivation and on-disk database key wrapping.

Protocol Flow



1. Pairwise Sessions (Double Ratchet Protocol):

- Initiated via an extended Triple Diffie-Hellman (3DH) handshake to derive a initial root key.
- Combines a symmetric-key ratchet (kDF updating per message) with a Diffie-Hellman ratchet (updating on each conversation turnaround) to guarantee Forward Secrecy and Break-in Recovery (Post-Compromise Security).

2. Group Channels (Megolm / TreeKEM Ratcheting):

- Each client maintains an independent outbound ratchet stream shared with channel members via pairwise sessions.
- Channel keys rotate automatically on a temporal interval or immediately whenever a user exits or is revoked from the room, preventing $O(N^2)$ pairwise re-encryption scaling costs.

3. Local Encrypted Vault:

- All long-term private identity keys, ratchet session parameters, and cached data are stored in a local SQLite file encrypted via **SQLCipher** or explicit page-level encryption using an Argon2id key derived from the user's master passphrase.

5. C-ABI Wrapper & Event Callback Interface

To prevent C++ name mangling and enable straightforward FFI bindings across standard programming languages, the core engine exposes a plain C interface API.

Core C API Structure

```
#ifndef ENGINE_CORE_H
```

```
#define ENGINE_CORE_H
```

```
#ifdef __cplusplus
```

```
extern "C" {
```

```
#endif
```

```
#if defined(_WIN32)
```

```
    #define EXPORT __declspec(dllexport)
```

```
#else
```

```
    #define EXPORT __attribute__((visibility("default")))
```

```
#endif
```

```
// Opaque handles hiding internal C++ object instances
```

```

typedef struct EngineContext EngineContext;

typedef struct ServerSession ServerSession;

// Callback signature for async events (messages, connection state changes)
typedef void (*EventCallback)(int event_type, const char* json_payload, void* user_data);

// Engine Lifecycle and Security Operations

EXPORT EngineContext* engine_create(const char* vault_path);

EXPORT int engine_unlock_vault(EngineContext* ctx, const char* passphrase);

EXPORT void engine_set_event_callback(EngineContext* ctx, EventCallback cb, void*
user_data);

// Network Operations

EXPORT ServerSession* engine_connect_server(EngineContext* ctx, const char*
server_url, const char* auth_key);

EXPORT int engine_send_message(EngineContext* ctx, const char* channel_id, const
char* plaintext);

EXPORT void engine_destroy(EngineContext* ctx);

#ifdef __cplusplus
}

#endif

#endif // ENGINE_CORE_H

```

Binding Integration Strategy

- **Python:** Bound via `ctypes` or `cffi`. Exposes an asynchronous Python interface converting raw C callback payloads into Python `asyncio` event loops.

- **Rust:** Uses `bindgen` to parse headers automatically or `cxx` to implement zero-cost memory mapping over core pointer allocations.
- **C# / Godot / .NET:** Utilizes `[DllImport]` or `LibraryImport` marshalling semantics, passing function pointers to match the `EventCallback` delegate model across managed heap boundaries.

6. Database Schemas (SQLite WAL)

Server Database Schema

-- User key identity table

```
CREATE TABLE users (
    user_id TEXT PRIMARY KEY,      -- Base64 encoded Ed25519 Public Key
    username TEXT NOT NULL,
    created_at INTEGER NOT NULL,
    access_level INTEGER DEFAULT 0
);
```

-- Channel & Room metadata

```
CREATE TABLE rooms (
    room_id TEXT PRIMARY KEY,
    room_name TEXT NOT NULL,
    topic TEXT,
    is_private INTEGER NOT NULL DEFAULT 1
);
```

-- Offline store-and-forward cipher store

```
CREATE TABLE messages_store (  
  
    message_id TEXT PRIMARY KEY,  
  
    room_id TEXT NOT NULL,  
  
    sender_id TEXT NOT NULL,  
  
    encrypted_payload BLOB NOT NULL,    -- Opaque ratchet payload  
  
    timestamp INTEGER NOT NULL,  
  
    FOREIGN KEY(room_id) REFERENCES rooms(room_id),  
  
    FOREIGN KEY(sender_id) REFERENCES users(user_id)  
  
);
```

Client Vault Schema (Encrypted On-Disk)

-- Known server connections

```
CREATE TABLE servers (  
  
    server_id TEXT PRIMARY KEY,  
  
    server_url TEXT NOT NULL,  
  
    auth_token TEXT NOT NULL,  
  
    nickname TEXT  
  
);
```

-- Active cryptographic session state (Double Ratchet)

```
CREATE TABLE ratchet_states (  
  
    session_id TEXT PRIMARY KEY,
```

```
peer_id TEXT NOT NULL,  
  
root_key BLOB NOT NULL,  
  
chain_key_send BLOB NOT NULL,  
  
chain_key_recv BLOB NOT NULL,  
  
next_dh_pubkey BLOB NOT NULL  
  
);  
  
-- Plaintext local message store (Decrypted)  
  
CREATE TABLE cached_messages (  
  
    msg_id TEXT PRIMARY KEY,  
  
    server_id TEXT NOT NULL,  
  
    channel_id TEXT NOT NULL,  
  
    sender_pubkey TEXT NOT NULL,  
  
    decrypted_text TEXT NOT NULL,  
  
    timestamp INTEGER NOT NULL,  
  
    FOREIGN KEY(server_id) REFERENCES servers(server_id)  
  
);
```

7. Implementation Roadmap

Phase 1: Core Networking & Server Foundation

- Implement asynchronous low-overhead socket server ([asio](#)).
- Establish binary protocol parser and SQLite WAL schema.
- Verify store-and-forward routing logic with basic load tests.

Phase 2: Client Core Engine Architecture

- Build C++ engine initialization loop with local storage interface.
- Implement `SQLCipher/Argon2id` local vault unlock pipelines.
- Set up state machine transitions for network connectivity and state sync.

Phase 3: Cryptography Engine Integration

- Integrate `libsodium` primitives for identity key generation (Ed25519) and key exchanges (X25519).
- Implement 3DH handshake and full Double Ratchet state transitions.
- Implement group channel ratcheting (Megolm pattern) for multi-party chat scalability.

Phase 4: C-ABI Standardization & TUI Validation

- Expose engine API through explicit C boundary interfaces (`extern "C"`).
- Develop a terminal frontend (TUI) utilizing `FTXUI` or `ncurses` linked dynamically against `libcore.so/libcore.dylib`.
- Validate memory isolation, runtime safety, and dynamic callback mechanics under stress testing.

Phase 5: Language Bindings & GUI Ecosystem

- Release reference FFI wrappers for Python (`ctypes`), Rust (`bindgen`), and C# (`P/Invoke`).
- Publish dynamic binding examples for desktop (Qt, Flutter) and mobile runtimes.
- Conduct formal cryptographic audits and benchmark protocol overhead across resource-constrained devices.